Intelligent Robotics (ASY2) · Lab 4

ICP

How far did the robot go? Scan matching, alone and with odometry

Duration 180 minutes
Script Cycle 4, section 4.1
Work in pairs
Files `ohm_icp/student/`

1 Overview

A robot drives a loop around a rack in the hall `production`, lap after lap (fig. 1). You do not steer it: a driver node does. Your node has to find out how the robot moved. It has two sensors:

- **Laser scanner**: 20 scans per second, 360 beams, up to 8 m. Each range scatters by 1.5 cm.
- **Odometry**: 50 messages per second from the wheels. Its wheel constants are a little wrong: every path is 3 % too long, every turn 3 % too large.

Two scans taken a moment apart see the same racks and walls, a little shifted and turned. The transform that maps the new scan onto the old one is the motion of the robot in between. ICP finds it. Chain all these motions and you have the pose of the robot: odometry from the laser.

- **Task 1**: ICP alone, after Besl and McKay. Each match starts at "the robot did not move".
- **Task 2**: ICP with the odometry: each match starts where the odometry says the robot went.

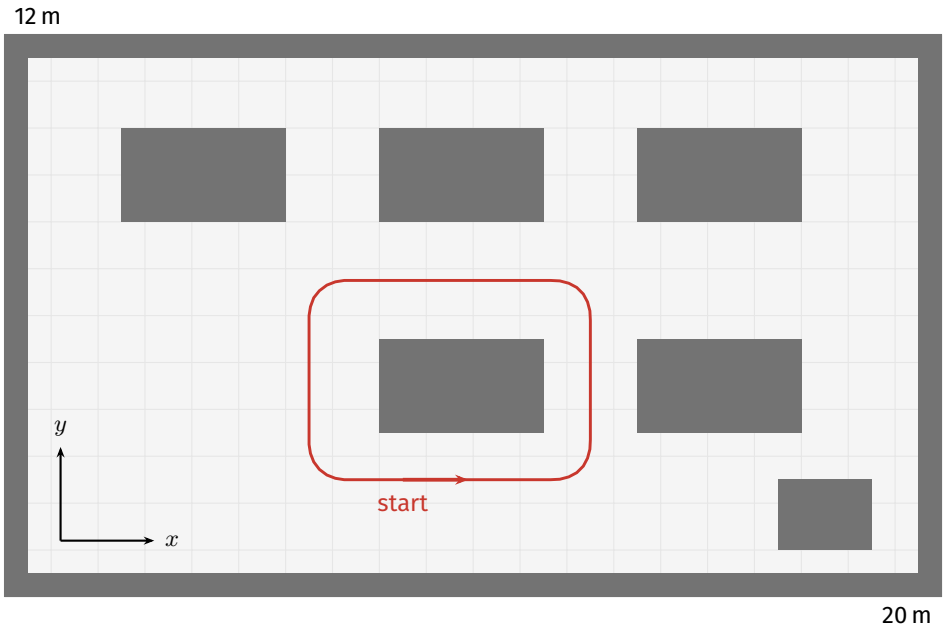


Figure 1: The hall `production` of the simulator from above, 20 m × 12 m. Dark: walls and racks. The robot drives the red loop counter-clockwise around the lower middle rack at 0.5 m/s: 19 m and 360 ° per lap, about 40 s.

Table 1: Schedule.

min	task	done when
0–15	section 3 The scan	table 2 is filled in
15–55	section 4 ICP on two point sets	`pytest -k task1` passes up to 1.5
55–75	section 5 ICP in the loop	table 3 is filled in
75–90	section 6 Pairs that do not belong	table 4 is filled in
90–110	section 7 How far apart?	table 5 is filled in
110–120	break	
120–165	section 8 ICP with odometry	table 6 is filled in
165–180	section 9 Extra task: the kd-tree	
	section 10 Extra task: every scan	if time is left

Notation as in the script. In the code the names are short:

script	code	
$M = \{\mathbf{m}_i\}$, $D = \{\mathbf{d}_j\}$	`M`, `D`	model (the older scan) and data (the newer one), one point per row
$\mathbf{c}_m$, $\mathbf{c}_d$	`c_m`, `c_d`	the centroids
M', D'	`Mc`, `Dc`	both sets shifted by their centroid
ϕ, $\mathbf{t}$, $\mathbf{T}$	`phi`, `t`, `T`	rotation, translation, the 3×3 transform
$\omega_{i,j}$	`pairs_kept`	the weight of a pair, 1 or 0
ϵ	`eps`	the stop: the error changed by less than this

Everything is in metres and radians. A pose is a 3×3 transform as $\mathbf{T}$: it maps a point from the frame of the robot into the hall (`map`).

2 Getting started

Your own copy. Fork `ohm_icp` on GitHub (button *Fork* at the top right), then clone your fork into the `src/` folder of your workspace. Update the simulator:

```
git clone git@github.com:<your-name>/ohm_icp.git
```

```
git -C ir-mecanum-lab pull
```

Commit after every task and push, so nothing is lost at the end of the session.

Build. Build once, then source the workspace in every new terminal:

```
colcon build --symlink-install --packages-select mecanum_lab_interfaces mecanum_lab
    ohm_icp
```

```
source install/setup.bash
```

You do not need to build again after a change to your code. Just start the node again.

Simulator. One launch file starts the simulator in the hall `production`, the driver, the monitor and RViz. The robot starts to drive at once.

```
ros2 launch ohm_icp icp.launch.py
```

The driver steers by the exact pose on `/muster/truth`. Your node never reads `/truth`: it has to find the motion from the scans (and the odometry) alone. The monitor compares with the truth.

Your files. `student/icp.py` (the algorithm) and `student/scan_odometry.py` (from the messages to the points, from the matches to the pose). You write only the few lines in the middle, marked

```
# the two centroids, and both sets shifted by their own centroid (eq. 4.5 to 4.8)
# TODO STUDENT 1.3
raise NotImplementedError("STUDENT 1.3")
```

The comment above it says in words what goes there; the equation is in the task of this handout. Replace the `raise` line by your code.

> **numpy** for the point sets: a set of N points is an $N \times 2$ array, one point per row. `M[:, 0]` is the column of all x, `M[:, 1]` of all y; `M.mean(axis=0)` the mean of every column; `M - c` subtracts `c` from every row; `a * b` multiplies element by element, `np.sum(a)` adds up; `A @ B` is the matrix product; `np.linalg.inv(A)` the inverse.

The node. `student/icp_node.py` is given. It subscribes to the scan and the odometry, calls your functions and publishes the pose. It runs in a second terminal:

```
ros2 run ohm_icp icp_node.py --ros-args -p task:=1
```

If a function is not written yet, the node stops with `NotImplementedError: STUDENT 1.1`: that is the next one to write.

Self check. The tests try your functions on made-up point sets, one task at a time:

```
python3 -m pytest test/test_icp.py -q -k task1
```

Three terminals. All of them in the workspace, sourced.

terminal 1	simulator, driver, monitor, RViz	the monitor prints here every 10 s
terminal 2	your node	Ctrl-C and start again after every change
terminal 3	tests, hints, plots, `ros2 topic`	

Every task starts with a box *Running*: what terminals 1 and 2 run. It ends with *What you should see*: the first lap and the monitor when the task is done. RViz uses the same colours: white the true path, blue the odometry, red your ICP. All three start where your node started.

The monitor and the plots. Each start of your node is a new run. Every 10 s the monitor prints, from the start of the run: how far the robot went (*driven*), how much it turned (*turned*, one lap is 360 °), and the error of the odometry and of ICP against the truth, in position and heading. After each lap it prints the errors again. The monitor writes every run to `runs/`; the last one as plots:

```
ros2 run ohm_icp plot_run.py
```

Hints. Stuck? Every block has three hints: the idea, how to write it in Python, and at last the code. Each call opens the next one, in the folder `ohm_icp`:

```
python3 hint.py 1.3
```

`python3 hint.py` alone shows the hints you have opened. Try on your own first: the hints notice when you ask too early.

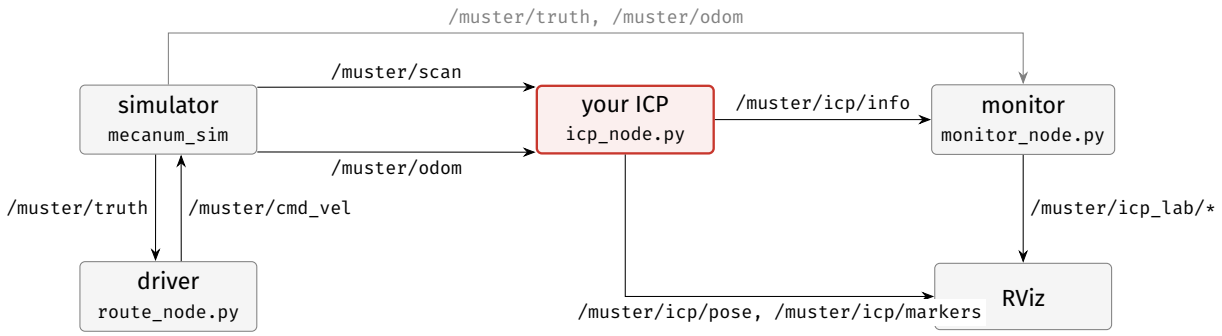


Figure 2: The nodes and topics of the lab. Red: your node. Grey: given.

3 Task 0: the scan

15 min

Running

terminal 1 `ros2 launch ohm_icp icp.launch.py`
terminal 2 `ros2 run ohm_icp icp_node.py --ros-args (nothing yet)`

Before you match scans, look at one. In terminal 3:

```
ros2 topic hz /muster/scan
```

```
ros2 topic echo --once /muster/scan
```

```
ros2 interface show sensor_msgs/msg/LaserScan
```

`echo` prints a long list of `ranges`. Add `--field angle_increment` (or another field) to see one field only. Fill in table 2.

Table 2: The laser scanner. Fill in.

	`/muster/scan`
message type	
scans per second	
frame of the points (`header.frame_id`)	
`angle_min`, `angle_increment` (in degrees)	
number of beams	
`range_min`, `range_max`	
the range of a beam that hits nothing	

- Beam 0 points forward (x of the robot). In which direction does beam 90 point?
- Watch the simulator window: the scan is drawn as dots. Which beams hit nothing on the loop?

Checkpoint A beam that hits nothing is not missing: it reads `range_max`. It is no point of a wall, and your code has to drop it.

4 Task 1: ICP on two point sets

40 min

Running

```
terminal 1   ros2 launch ohm_icp icp.launch.py
terminal 2   ros2 run ohm_icp icp_node.py --ros-args (nothing yet, the tests first)
```

First the algorithm itself, without ROS: `icp.py`, the four steps of the script.

Lecture script, section 4.1.3, ICP after Besl and McKay

1. For each point $\mathbf{d}_j \in D$ find the closest point $\mathbf{m}_i \in M$.
2. From these pairs compute $\mathbf{R}$ and $\mathbf{t}$ that minimise

$$E(\mathbf{R}, \mathbf{t}) = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{m}_i - (\mathbf{R}\mathbf{d}_i + \mathbf{t})\|^2. \tag{4.4}$$

3. Apply the transform to D.
4. If the error changed by less than ϵ, stop; otherwise go to 1.

Step 2 in closed form. The centroids, and both sets shifted by them:

$$\mathbf{c}_m = \frac{1}{N} \sum_{i=1}^{N} \mathbf{m}_i, \quad \mathbf{c}_d = \frac{1}{N} \sum_{i=1}^{N} \mathbf{d}_i, \qquad \mathbf{m}'_i = \mathbf{m}_i - \mathbf{c}_m, \quad \mathbf{d}'_i = \mathbf{d}_i - \mathbf{c}_d. \tag{4.5–4.8}$$

The rotation and the translation:

$$\phi = \operatorname{atan2}\Big(\sum_{i=1}^{N} \big(y'_{m,i}\, x'_{d,i} - x'_{m,i}\, y'_{d,i}\big), \sum_{i=1}^{N} \big(x'_{m,i}\, x'_{d,i} + y'_{m,i}\, y'_{d,i}\big) \Big), \tag{4.14}$$

$$\mathbf{t} = \mathbf{c}_m - \mathbf{R}(\phi)\, \mathbf{c}_d, \tag{4.17}$$

and as one transform $\mathbf{T} = \begin{pmatrix} \cos\phi & -\sin\phi & t_x \\ \sin\phi & \cos\phi & t_y \\ 0 & 0 & 1 \end{pmatrix}$ (4.18).

STUDENT 1.2 `closest_points(D, M)`, step 1, brute force: for every row of `D` the index of the closest row of `M` and its distance. `diff`, all differences $\mathbf{d}_j - \mathbf{m}_i$, is given.
`python3 hint.py 1.2`

STUDENT 1.3 `best_transform(M, D)`: the centroids and the shifted sets (4.5–4.8). Row i of `M` and of `D` is pair i.
`python3 hint.py 1.3`

STUDENT 1.4 `best_transform`: ϕ with (4.14) and $\mathbf{t}$ with (4.17). `rotation(phi)` and `se2(x, y, phi)` are given.
`python3 hint.py 1.4`

STUDENT 1.5 `icp(M, D)`, step 3: move `D_now` by the step `T_step`, and put `T_step` on top of the transform so far, `T`. Steps 1, 2 and 4 are given around it.
`python3 hint.py 1.5`

The order of the product. ICP finds $\mathbf{T}_1$ in the first iteration, applies it, then finds $\mathbf{T}_2$ for the points that already moved. A point of D ends at $\mathbf{T}_2\mathbf{T}_1\mathbf{d}$: the newest step stands on the *left*. After n iterations $\mathbf{T} = \mathbf{T}_n \cdots \mathbf{T}_2\mathbf{T}_1$.

`pytest -k task1` also tries the exercise of the script ($M = \{(5,4),(6,2)\}$, $D = \{(1.5,2.7),(2,0.5)\}$ gives $\phi = 13.76°$ and $\mathbf{t} = (4.18, 1.03)$) and a room with a box in it, moved and turned. Tests 1.1, 1.6 and 1.7 fail until the next tasks.

Checkpoint `pytest -k "task1 and not task1_1 and not task1_6 and not task1_7"` passes. Why is one iteration enough for the exercise of the script, and not for the room?

5 Task 1: ICP in the loop

20 min

Running

```
terminal 1   ros2 launch ohm_icp icp.launch.py
terminal 2   ros2 run ohm_icp icp_node.py --ros-args -p task:=1 -p d_max:=0
```

Now the scans of the robot. The node keeps the last matched scan as the model M. Every tenth scan (twice a second) becomes the data D: ICP maps it onto the model, the result $\mathbf{T}$ is the motion of the robot in between. Then the new scan is the model for the next match.

STUDENT 1.1 `scan_odometry.py`, `scan_to_points(msg)`: the points of one scan in the frame of the laser, $(r\cos\alpha,\ r\sin\alpha)$ with α = `angle_min` + $i\cdot$ `angle_increment`. Drop every beam without an echo (table 2). `python3 hint.py 1.1`

$\mathbf{T}$ maps the new scan onto the old one: it is the pose of the robot now, in the frame of the robot at the old scan. The pose at the old scan maps that frame into the hall.

STUDENT 1.6 `next_pose(pose, T)`: the pose at the new scan, from the pose at the old scan and $\mathbf{T}$. Both are 3×3 transforms. `python3 hint.py 1.6`

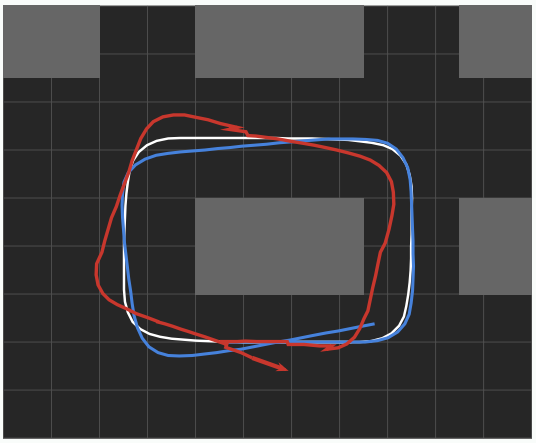
Start the node. It starts where the odometry is. Watch RViz: grey the model, amber the new scan where ICP starts, red the new scan where ICP put it, light blue the pairs. Let the robot drive one full lap (≈ 40 s) and take the line `lap 1` of the monitor. Fill in table 3.

Table 3: One lap, ICP alone with `d_max:=0`. Fill in.

	driven in m	turned in °	error in m	error in °
truth		360	—	—
odometry				
ICP				

- Which of the two is better in heading? Which one in distance?
- In RViz, watch the pairs while the robot passes a rack. Do all of them connect the same wall?

What you should see



`d_max:=0`; the first lap after the start of the node: white the truth, blue the odometry, red ICP. Terminal 1, the monitor (the numbers change from run to run, the ms with the computer):

```
t   10.1 s | driven: truth   4.5 m, odom   4.7 m, ICP   3.7 m | turned: truth   88, odom   92, ICP
   88 deg | error: odom 0.11 m  3.1 deg, ICP 0.91 m 10.0 deg
t   20.1 s | driven: truth   9.4 m, odom   9.7 m, ICP   8.6 m | turned: truth  180, odom  187, ICP
   182 deg | error: odom 0.18 m  5.4 deg, ICP 0.21 m  8.7 deg
t   30.1 s | driven: truth  14.2 m, odom  14.7 m, ICP  13.4 m | turned: truth  270, odom  281, ICP
   278 deg | error: odom 0.50 m  8.3 deg, ICP 0.70 m 13.5 deg
t   40.1 s | driven: truth  19.1 m, odom  19.7 m, ICP  17.7 m | turned: truth  360, odom  375, ICP
   367 deg | error: odom 0.45 m 11.1 deg, ICP 1.51 m 17.2 deg
lap 1 (19 m): odom 0.45 m 10.9 deg | ICP 1.59 m 17.3 deg | 81 matches, 22 iterations and 74 ms per
   match
```

Checkpoint The ICP path has about the right shape, but it is too short, and its heading is no better than the one of the odometry. The next task finds out why.

6 Task 1: pairs that do not belong

15 min

Running

terminal 1 `ros2 launch ohm_icp icp.launch.py`
terminal 2 `ros2 run ohm_icp icp_node.py --ros-args -p task:=1` , then `-p d_max:=1.0`

The new scan sees things the old one did not: the side of a rack that came into view, a wall that was out of range. Step 1 still finds a closest point for each of them, a wrong pair. Wrong pairs pull the result off. In (4.3) of the script every pair has a weight $\omega_{i,j}$, 1 or 0. Give the weight 0 to every pair farther apart than `d_max`.

STUDENT 1.7 `icp.py`, `pairs_kept(distance, d_max)`: `True` for every pair whose distance is at most `d_max`.

`python3 hint.py 1.7`

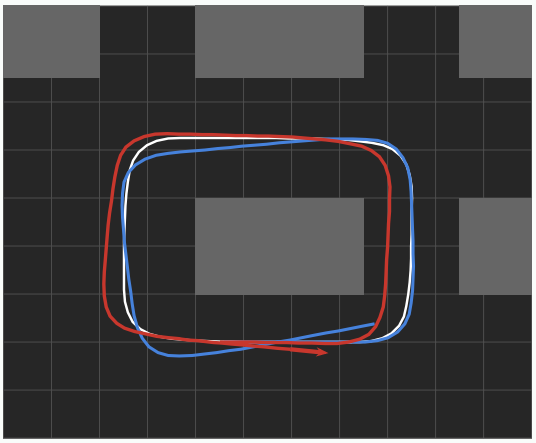
The default is `d_max` = 0.3 m. Run one lap with it, then one with 1.0 m. Fill in table 4 (the row 0 is from table 3).

Table 4: One lap, ICP alone, three values of `d_max`. Fill in.

`d_max` in m	driven in m	turned in °	error in m	error in °
0 (all pairs)				
0.3				
1.0				

- Why does 1.0 m help less than 0.3 m?
- The robot moves 25 cm between two matches on a straight. What would happen with `d_max` = 0.1 m? Try it.

What you should see



`d_max:=0.3`; the first lap after the start of the node: white the truth, blue the odometry, red ICP. Terminal 1, the monitor (the numbers change from run to run, the ms with the computer):

```
t   10.1 s | driven: truth   4.5 m, odom   4.7 m, ICP   4.0 m | turned: truth   88, odom   92, ICP
   89 deg | error: odom 0.11 m  3.1 deg, ICP 0.49 m  0.7 deg
t   20.1 s | driven: truth   9.4 m, odom   9.7 m, ICP   8.9 m | turned: truth  180, odom  187, ICP
   180 deg | error: odom 0.18 m  5.4 deg, ICP 0.41 m  2.1 deg
t   30.1 s | driven: truth  14.2 m, odom  14.7 m, ICP  13.6 m | turned: truth  270, odom  281, ICP
   269 deg | error: odom 0.50 m  8.3 deg, ICP 0.31 m  2.8 deg
t   40.1 s | driven: truth  19.1 m, odom  19.7 m, ICP  17.9 m | turned: truth  360, odom  375, ICP
   359 deg | error: odom 0.45 m 11.1 deg, ICP 0.96 m  4.4 deg
lap 1 (19 m): odom 0.45 m 10.9 deg | ICP 1.04 m  4.4 deg | 81 matches, 28 iterations and 88 ms per
   match
```

Checkpoint `pytest -k task1` passes. With `d_max` = 0.3 m ICP alone is better than the odometry in heading by more than a factor two. In position it is not better: its path is still too short.

7 Task 1: how far apart?

20 min

>_ Running

terminal 1 `ros2 launch ohm_icp icp.launch.py`
terminal 2 `ros2 run ohm_icp icp_node.py --ros-args -p task:=1 -p every:=2` , then 5 and 20

`every` says which scans are matched: 10 is every tenth scan, twice a second. Try 2, 5 and 20, one lap each (table 5, the row 10 is from table 4).

Table 5: One lap, ICP alone, `d_max` = 0.3 m. Fill in. ms: per match, terminal 2.

`every`	driven in m	turned in °	error in m	error in °	ms
2					
5					
10					
20					

- `every` = 2: the path is far too short. Each match finds a little too little motion: the beams of both scans hit the wall at almost the same places, and ICP stops where every point sits on its old neighbour, a local minimum. What does that small error per match do when there are five times as many matches?
- `every` = 20: half a metre and up to 36 ° in a corner between two scans. Look at the first match after a corner in RViz (fig. 3, left). Why does ICP fall into the wrong minimum here? What does `d_max` do to the right pairs, which are now more than 0.3 m apart?

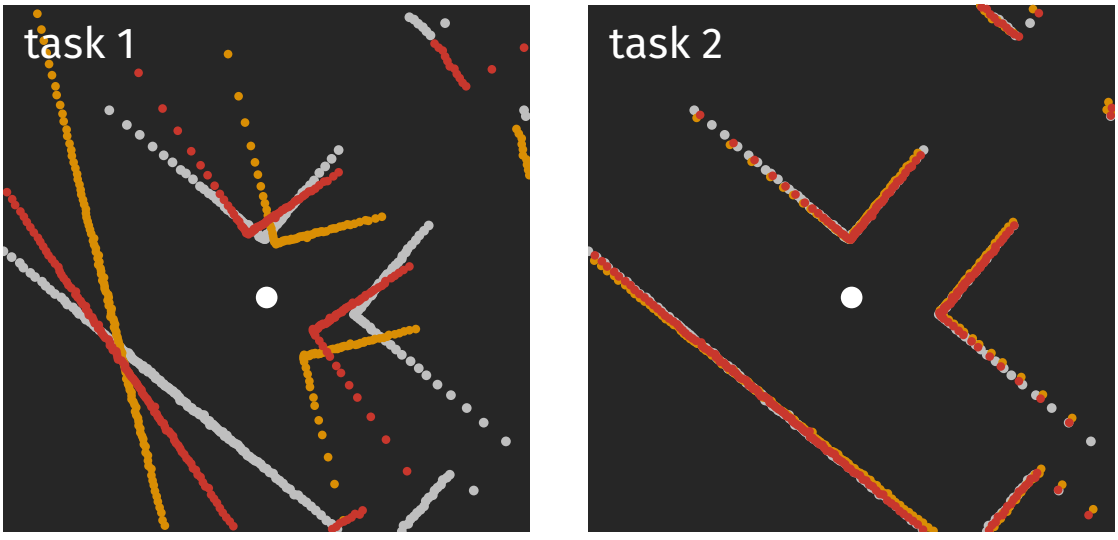
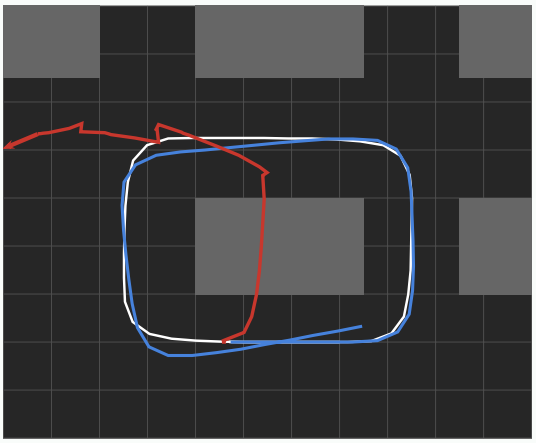


Figure 3: One match 20 scans apart in a corner, in the frame of the robot at the old scan (white dot). Grey the model, amber the new scan where ICP starts, red where ICP put it. Left, task 1, from the identity: 0.5 m off. Right, task 2, from the odometry: 1 cm off.

What you should see



`every:=20`; the first lap after the start of the node: white the truth, blue the odometry, red ICP. Terminal 1, the monitor (the numbers change from run to run, the ms with the computer):

```
t   10.1 s | driven: truth   4.5 m, odom   4.7 m, ICP   1.4 m | turned: truth   88, odom   92, ICP
   87 deg | error: odom 0.11 m  3.1 deg, ICP 3.16 m  1.9 deg
t   20.1 s | driven: truth   9.4 m, odom   9.7 m, ICP   5.7 m | turned: truth  180, odom  186, ICP
   162 deg | error: odom 0.18 m  5.4 deg, ICP 2.46 m 19.5 deg
t   30.1 s | driven: truth  14.2 m, odom  14.7 m, ICP   7.4 m | turned: truth  270, odom  279, ICP
   177 deg | error: odom 0.50 m  8.3 deg, ICP 1.43 m 94.7 deg
t   40.1 s | driven: truth  19.1 m, odom  19.7 m, ICP   9.6 m | turned: truth  360, odom  372, ICP
   207 deg | error: odom 0.45 m 11.1 deg, ICP 6.07 m 157.1 deg
lap 1 (19 m): odom 0.46 m 10.9 deg | ICP 6.44 m 157.2 deg | 41 matches, 26 iterations and 94 ms per
   match
```

Checkpoint Explain with table 5: too close and the small errors of many matches add up; too far and ICP starts outside the valley of the right minimum. The script calls this convergence to a *local* minimum.

8 Task 2: ICP with odometry

45 min

Running

terminal 1 `ros2 launch ohm_icp icp.launch.py`
terminal 2 `ros2 run ohm_icp icp_node.py --ros-args -p task:=2 -p every:=2` , then 5, 10 and 20

ICP alone starts every match at the identity, as if the robot had not moved. The odometry knows better: it measured how far the robot went and how much it turned. Start ICP there. The odometry gives two poses in its own frame, $\mathbf{T}_{\text{before}}$ at the old scan and $\mathbf{T}_{\text{now}}$. The motion in between, in the frame of the robot at the old scan:

$$\mathbf{T}_0 = \mathbf{T}_{\text{before}}^{-1}\,\mathbf{T}_{\text{now}}.$$

`icp(M, D, T_init)` moves D by $\mathbf{T}_0$ first and starts the product with it. The rest of ICP stays as it is.

STUDENT 2.1 `scan_odometry.py`, `odom_guess(odom_prev, odom_now)`: $\mathbf{T}_0$ from two `nav_msgs/msg/Odometry`. `se2` and `yaw` are given. `python3 hint.py 2.1`

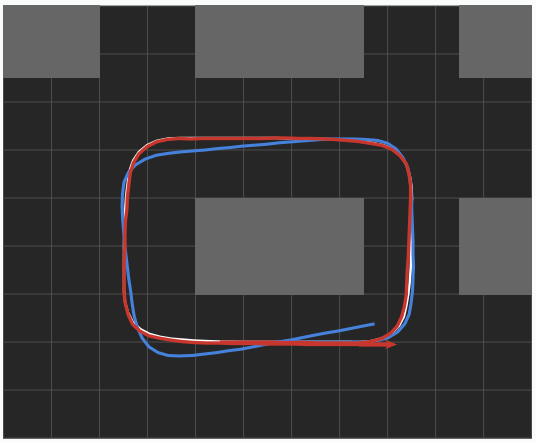
Start the node with `task:=2` and try the values of table 6, one lap each.

Table 6: One lap, ICP with the odometry as the start. Fill in. ms: per match, terminal 2.

`every`, `d_max`	driven in m	turned in °	error in m	error in °	ms
2, 0.3					
5, 0.3					
10, 0.3					
20, 0.3					
10, 0					

- `every` = 20 failed in task 1. Why does it work now (fig. 3, right)? Why does ICP need fewer iterations?
- The odometry drives 3 % too far. ICP starts from it. Does the ICP path come out 3 % too long?
- `every` = 2 is still too short, even from the odometry. Why does a good start not help here?
- Without `d_max` (last row) ICP is about as bad in heading as the odometry it started from, or worse. Why?
- Which sensor knows the distance well, which the heading? Which `every` do you keep?

What you should see



`every:=10`; the first lap after the start of the node: white the truth, blue the odometry, red ICP. Terminal 1, the monitor (the numbers change from run to run, the ms with the computer):

```
t   10.1 s | driven: truth   4.5 m, odom   4.7 m, ICP   4.5 m | turned: truth   88, odom   92, ICP
   89 deg | error: odom 0.11 m  3.1 deg, ICP 0.04 m  0.2 deg
t   20.1 s | driven: truth   9.4 m, odom   9.7 m, ICP   9.4 m | turned: truth  180, odom  187, ICP
   181 deg | error: odom 0.18 m  5.4 deg, ICP 0.03 m  0.2 deg
t   30.1 s | driven: truth  14.2 m, odom  14.7 m, ICP  14.2 m | turned: truth  270, odom  281, ICP
   272 deg | error: odom 0.50 m  8.3 deg, ICP 0.07 m  0.4 deg
t   40.1 s | driven: truth  19.1 m, odom  19.7 m, ICP  19.1 m | turned: truth  360, odom  375, ICP
   362 deg | error: odom 0.45 m 11.1 deg, ICP 0.04 m  0.0 deg
lap 1 (19 m): odom 0.45 m 10.9 deg | ICP 0.04 m  0.1 deg | 81 matches, 9 iterations and 30 ms per
   match
```

Checkpoint `pytest -k task2` passes. After one lap ICP with odometry is more than five times better than the odometry, in position and in heading. Show the plots of `plot_run.py` for task 1 and task 2.

9 Extra task: the kd-tree

15 min

Running

```
terminal 1   ros2 launch ohm_icp icp.launch.py
terminal 2   ros2 run ohm_icp icp_node.py --ros-args -p task:=1 -p kdtree:=true
```

Step 1 is the expensive one. Brute force compares every point of D with every point of M: $\mathcal{O}(N_d N_m)$ per iteration, here about 300×300. A kd-tree of M is built once per match, $\mathcal{O}(N_m \log N_m)$; then each query costs $\mathcal{O}(\log N_m)$ (script, section 4.1.3). `scipy.spatial.cKDTree` is such a tree. With `-p kdtree:=true` the node builds it and calls your function instead of `closest_points`.

STUDENT Z.1 `icp.py`, `closest_points_kdtree(D, tree)`: the same `(index, distance)` as `closest_points`, with one call of `tree.query(D)`. `python3 hint.py Z.1`

- Compare the ms per match in terminal 2 with table 5 (`every` = 10). How many times faster? Is the pose the same?
- Brute force needs $N_d N_m$ distances per iteration. How many is that for two scans of 300 points? And for two 3D scans of 100 000 points each?

10 Extra task: every scan

if time is left

>_ Running

```
terminal 1   ros2 launch ohm_icp icp.launch.py
terminal 2   ros2 run ohm_icp icp_node.py --ros-args -p task:=1 -p kdtree:=true -p
             keyframe:=true
```

In section 7 the small steps failed: `every` = 2 chains five times as many matches as 10, and the small error of each match adds up. So do not chain every scan onto the one before. Keep one scan as the model, the *keyframe*, and match every new scan onto it, 20 times a second. Only when the robot is `key_dist` away from the keyframe or turned by `key_angle`, the newest scan becomes the new keyframe. A pose 20 times a second, but a new link in the chain only every half metre:

$$\mathbf{T}_{\text{robot}} = \mathbf{T}_{\text{keyframe}}\,\mathbf{T}, \qquad \mathbf{T} \text{ maps the newest scan onto the keyframe.}$$

In task 1 ICP starts at the last match onto the same keyframe: the robot moved only 2.5 cm since. The node does the bookkeeping. It needs the kd-tree of section 9: brute force takes longer than the 50 ms between two scans.

STUDENT Z.2 `scan_odometry.py`, `new_keyframe(T, key_dist, key_angle)`: `True` when the robot is farther than `key_dist` from the keyframe or turned by more than `key_angle` (radians), in either direction. `python3 hint.py Z.2`

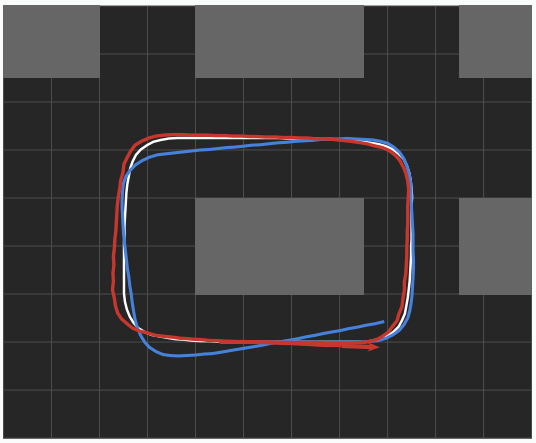
The default is 0.5 m and 20 °. Run one lap each with the values of table 7 (`-p key_dist:=1.0 -p key_angle:=30.0`).

Table 7: One lap, every scan onto the keyframe, ICP alone. Fill in. Keyframes: terminal 2.

`key_dist`, `key_angle`	driven in m	turned in °	error in m	error in °	keyframes
0.5, 20					
1.0, 30					
3.0, 90					
5.0, 180					

- Compare with table 5. Why is the path no longer too short?
- In section 7 half a metre between two scans was too much for ICP alone. Now 1 m works. What is different? (Hint: fig. 3.)
- Why does 3 m still work, and 5 m and 180 ° not? What must the keyframe and the newest scan have in common?

What you should see



`keyframe:=true`; the first lap after the start of the node: white the truth, blue the odometry, red ICP. Terminal 1, the monitor (the numbers change from run to run, the ms with the computer):

```
t   10.1 s | driven: truth   4.4 m, odom   4.5 m, ICP   4.3 m | turned: truth   86, odom   90, ICP
  87 deg | error: odom 0.11 m  3.1 deg, ICP 0.11 m  0.5 deg
t   20.1 s | driven: truth   9.2 m, odom   9.5 m, ICP   9.1 m | turned: truth  180, odom  187, ICP
  181 deg | error: odom 0.18 m  5.4 deg, ICP 0.10 m  1.6 deg
t   30.1 s | driven: truth  14.1 m, odom  14.5 m, ICP  14.0 m | turned: truth  270, odom  281, ICP
  271 deg | error: odom 0.50 m  8.3 deg, ICP 0.17 m  2.3 deg
t   40.1 s | driven: truth  19.0 m, odom  19.6 m, ICP  18.8 m | turned: truth  360, odom  375, ICP
  362 deg | error: odom 0.45 m 11.1 deg, ICP 0.37 m  2.0 deg
lap 1 (19 m): odom 0.45 m 11.0 deg | ICP 0.33 m  2.0 deg | 812 matches, 12 iterations and 3 ms per
  match
```

11 Outlook: a map

Your ICP chains scan to scan, or keyframe to keyframe. Each link adds a small error, and nothing ever takes it back: it is odometry, only a much better one. Match the scan against a *map* instead, built from all scans so far, and the error stops growing while the robot stays in known places. When it comes back after a long loop and the scan does not fit the map, the robot has found its own drift: that is loop closure, and with it SLAM (script, sections 4.2 to 4.4).